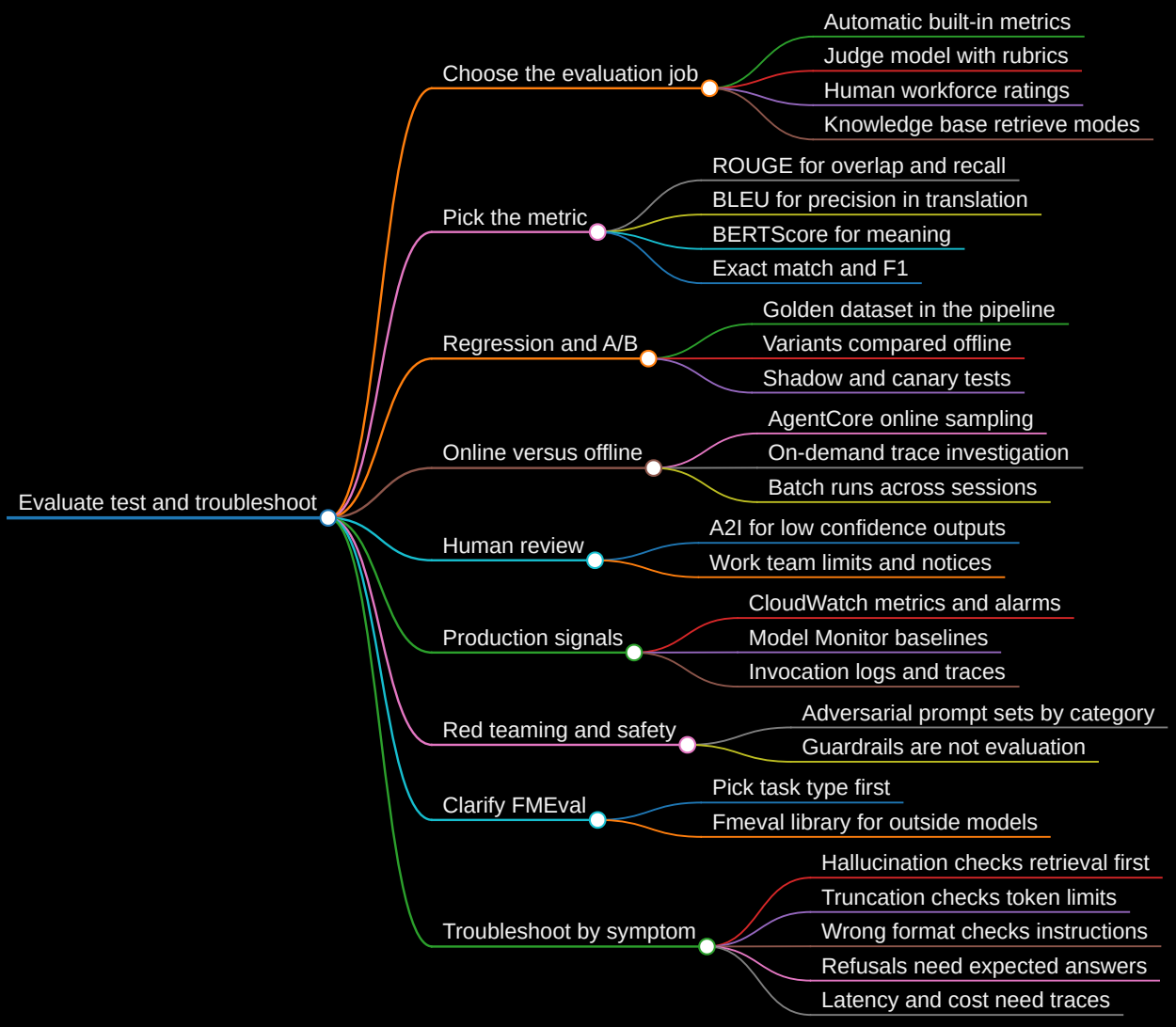
m8: Evaluate, test, and troubleshoot generative AI output

After this module, you can read a long scenario about quality, testing, or production behavior and name the deciding constraint. You can then pick the feature that fits and reject each wrong option with one reason. The module covers model evaluation jobs, knowledge base evaluation, the main metrics, golden datasets, prompt A/B tests, online and offline agent evaluation, human review, CloudWatch signals, and troubleshooting by symptom. Read the map first, then read the sections that match the services in your weak list.



The deciding constraint

Start with the object under test. A model output, a RAG answer, and an agent session each point to different features. After that, find the signal that decides the rest. Most wrong options solve a different problem, such as runtime safety, infrastructure health, or cost.

Rule: Decide what is being scored before you decide which service to use. Name the object, the timing, and the evidence type first.

Three words decide most questions in this area. The first is timing: before release, or after release on live traffic. The second is evidence: a reference answer, a rubric, a human opinion, or a retrieved passage. The third is the object: a model, a knowledge base, or an agent that calls tools.

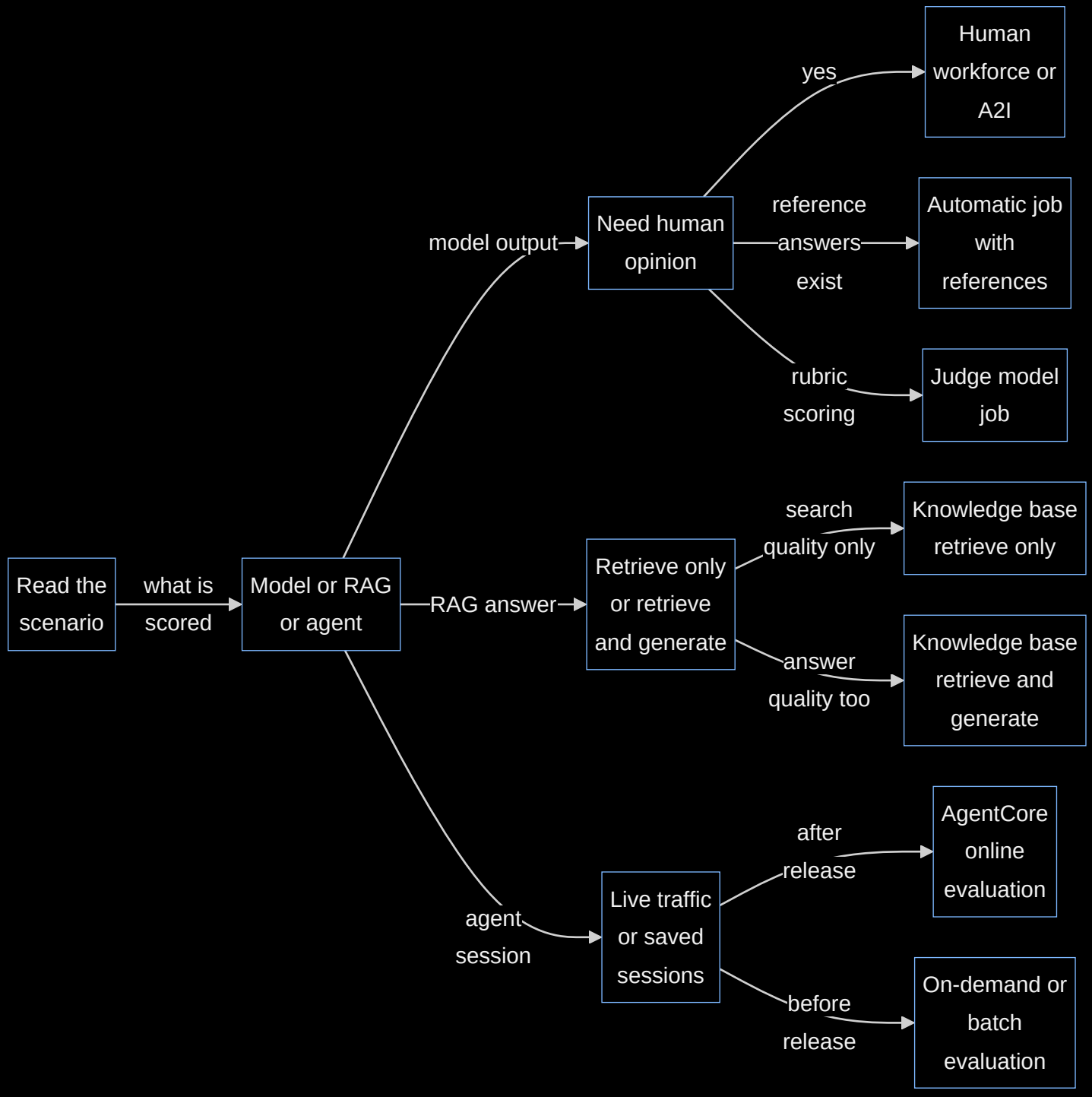


Figure: The first split is the object, then the evidence type and timing.

Use these signal words to move down each branch. A signal word is a phrase that points to one branch. Underline it in your head before you read the options.

- Reference answers, labeled answers, ground truth: an automatic job that compares to a reference.
- Rubric, scale, judge, explanation of the score: a judge model job.
- Reviewers, side by side, staff ratings: a human evaluation job or A2I, depending on live or offline.

- Retrieved passages, chunks, search results: a retrieve-only job or a retrieval test.
- Unsupported claims, fabricated facts, grounded in sources: faithfulness on a retrieve and generate job.
- Live sessions, sampling rate, production traffic: an online evaluation.
- Before promotion, release gate, pipeline: a golden dataset run in CI/CD.
- Feature drift, baseline, monitoring schedule: SageMaker Model Monitor, which watches statistics, not text quality.
- Bias in training data, class imbalance, facets: Clarify bias metrics, which work on datasets, not on generated text.

Services and features in scope

Bedrock model evaluation jobs

A Bedrock model evaluation job scores model responses to a prompt dataset.

The job has a generator model, which writes responses, and a scoring method. Scoring can use built-in metrics, custom metrics, a judge model, or a human work team. The report goes to an S3 bucket that you choose.

The automatic job type needs at least one Bedrock foundation model, a service role, and S3 input and output in the same Region as the job. The source states that CORS is not required for automatic jobs. CORS is needed only for human-based jobs.

Rule: Choose a Bedrock evaluation job when you need repeatable scores for models or prompt versions on your own dataset, and you want no scoring code.

The generator does not have to be a Bedrock base model. The source lists customized models, imported models, Provisioned Throughput models, prompt routers, and Bedrock Marketplace models as generators. You can also supply responses from a model outside Bedrock in the dataset. In that case, the job skips the invoke step and scores your responses directly.

Trap: A Bedrock job scores a fixed dataset. It does not watch live users. Pick a production tool when the question is about what users see today.

Pairs with: S3 for the dataset and report, Prompt Management variants for the prompt versions, and judge models for scoring. **Limits and numbers:** the per-job prompt limit is Not in source for this review. The judge model list changes over time, so check it when you run the job.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation-judge.html and https://docs.aws.amazon.com/bedrock/latest/userguide/model-evaluation-type-automatic.html

LLM as a judge and custom metrics

A judge job pairs a response writer with a separate scoring model.

The evaluator reads each prompt and response pair, then scores it with a metric prompt. The console shows a score histogram and explanations for the first five prompts. The full report is in S3.

Built-in judge metrics include Correctness, Completeness, Faithfulness, Helpfulness, Logical coherence, Relevance, Following instructions, Professional style and tone, Harmfulness, Stereotyping, and Refusal. Correctness and Completeness use a reference response when you supply one. Faithfulness checks whether the response adds facts that are not in the prompt context.

Rule: Use a judge metric when the quality needs a rubric and no exact answer string exists. Use Faithfulness when the question is whether the answer stays inside the supplied context.

A custom metric is a prompt that you write for the evaluator. Use one when your business rule is not in the built-in list, such as a tone rule for a brand. The source lists a different set of evaluator models for custom metrics than for built-in metrics. Check both lists when you pick a judge.

Trap: A judge score is not a fact check against the outside world. Faithfulness measures agreement with the context you supplied. A fluent wrong fact can still pass if the context also says it.

Trap: The judge can share the blind spots of the generator. The course material warns that a judge with the same problems as the model under test gives weak evidence.

Pairs with: Bedrock evaluation jobs, AgentCore evaluators (same judge idea, but for agents), and Prompt Management variants. **Limits and numbers:** the score scale for each built-in metric is Not in source for this review.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/model-evaluation-metrics.html and https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation-judge.html and course slide extract, pages 423 and 430

Bedrock human-based evaluation jobs

A human-based job uses a work team of people that you choose.

The work team can come from an Amazon Cognito user pool, a SageMaker Ground Truth private workforce, or Amazon Augmented AI. Each team supports up to 50 workers. You can enter up to 50 email addresses at one time.

Comparative rating methods show two or more responses and ask which is better. They include Likert comparison, choice buttons, and ordinal rank. Individual rating methods score one response at a time, using thumbs up or down, or a Likert scale. A single-model job must use an individual method.

Rule: Choose a human job when quality depends on tone, domain taste, or expert judgment that a metric or judge cannot score well.

Workers are notified of a job only when you first add them to the team. For a later job with the same workers, you must tell them directly and give them the portal link. Plan that step into the release schedule.

Trap: A human job is slow and does not scale to every request. It is a release-time or periodic check, not a per-request safety control.

Pairs with: A2I for live review, SageMaker Ground Truth for workforce management, and Cognito for user pools. **Limits and numbers:** the maximum number of models in one human job is Not in source for this review.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/human-worker-evaluations.html and course slide extract, page 421

Knowledge base and RAG evaluation jobs

A RAG evaluation job is either retrieve only or retrieve and generate.

A retrieve-only job reports on the passages returned from the knowledge base or an external RAG source. A retrieve-and-generate job also reports on the answer that the generator wrote. Both types can use a Bedrock knowledge base or your own response data.

The course material places retrieval metrics such as relevance and coverage in the retrieve-only type. It places answer metrics such as correctness, completeness, helpfulness, and logical coherence in the retrieve-and-generate type. The exact RAG metric identifiers were not confirmed from the AWS page in this review, except Builtin.Faithfulness and Builtin.Correctness, which appear on the judge metric list.

Rule: When an answer is wrong, test retrieval first. If the right passage is missing, a better generator cannot fix the answer.

A retrieve-only job does not test the generator model. If you change the generation model or its prompt, run a retrieve-and-generate job. If you change the chunking or the search settings, run a retrieve-only job first to see the effect on passages.

Trap: A faithfulness score on a retrieve-and-generate job does not show that the search returned the best passages. Check retrieval separately.

Trap: A retrieve-only job does not show whether the final answer is grounded. It stops before generation.

Pairs with: the Knowledge Bases retrieval interface for your own test scripts, S3 for the dataset, and CloudWatch for trends. **Limits and numbers:** the prompt

limit per RAG job is Not in source for this review.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation-kb.html and course slide extract, page 430

Overlap and similarity metrics

Each metric below measures a different thing. The job type and the task type decide which one is valid. Read the task first. A summary task, a translation task, a short answer task, and a label task use different metrics.

ROUGE

ROUGE is a set of metrics for text summarization and machine translation. It counts overlapping units between the generated text and the reference. ROUGE-N counts n-gram overlap. ROUGE-L uses the longest common subsequence, which captures order. ROUGE is recall-oriented.

Rule: Use ROUGE when a reference summary exists and word overlap is the goal. Use ROUGE-L when the order of ideas matters.

Trap: ROUGE rewards shared words. A correct paraphrase can score low, and a wrong sentence with the same words can score high.

BLEU

BLEU is used for machine translation. It measures precision of n-grams against one or more human translations. It works at sentence level and has a brevity penalty for overly short outputs. The course material says it has limited use for fluency and grammar.

Rule: Use BLEU for translation checks that have human reference translations.

Trap: BLEU is a weak choice for open-ended generation. It does not check meaning, and it penalizes valid short answers.

BERTScore

BERTScore compares embedding vectors of the generated text and the reference text. It measures semantic similarity, so it is less sensitive to synonyms and paraphrase. The AWS overview of summarization accuracy lists

BERTScore next to ROUGE-N and Meteor, and it uses a large NLI-trained model to build the embeddings.

Rule: Use BERTScore when meaning matters more than exact wording, and a reference exists.

Trap: A high semantic score does not prove the facts are right. A number change in a sentence can still leave the embedding similar.

F1, exact match, and accuracy

For question answering, the AWS overview lists exact match, quasi-exact match after normalization, and F1 over words. F1 is the harmonic mean of precision and recall over the words of the answer. For classification, it lists classification accuracy, precision, recall, and balanced classification accuracy. Precision and recall can be averaged in several ways across classes.

Rule: Use exact match or F1 for short answers with a known string. Use precision when false positives cost more. Use recall when false negatives cost more.

Trap: F1 over words is not a meaning test. A short answer with the right words in the wrong order can still score well.

Toxicity

The AWS overview describes toxicity scoring with detector models. One detector returns seven scores for classes such as toxicity, insult, threat, and identity attack. The other returns a binary label trained on implicit toxicity. Built-in datasets include RealToxicityPrompts and BOLD.

Rule: Use a toxicity detector to screen outputs against an agreed threshold, and report the class scores, not only one number.

Trap: A toxicity score does not show bias or factual accuracy. A polite answer can still be wrong or unfair.

Robustness

Semantic robustness measures how much the score changes when the input changes but the meaning does not. The perturbations include keyboard typos,

random uppercase changes, and extra or missing spaces. The AWS overview defines it as the absolute difference in accuracy between the clean and the perturbed input.

Rule: Use the robustness metric when real users will send noisy text, such as chat messages with typos.

Trap: Robustness does not show whether the answer is correct on clean input. A model can be stable and always wrong.

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-foundation-model-evaluate-overview.html and https://docs.aws.amazon.com/bedrock/latest/userguide/model-evaluation-metrics.html and course slide extract, pages 425 to 427

Golden datasets and regression testing

A golden dataset is a fixed set of test cases with expected results.

Each case has an input, and often a reference answer or an expected label. You run every candidate against the same cases, then compare the scores with the last approved run. The course material says unit tests alone often miss GenAI behavior because the output is not deterministic.

The course material lists CloudWatch synthetic canaries, Step Functions, EventBridge, and Lambda as tools for end-to-end checks before and after a deployment. It suggests storing the results in S3 and analyzing them with Athena and QuickSight. Step Functions adds auditable state transitions, retries, and explicit approval steps.

Rule: Put the golden dataset run into the CI/CD pipeline, and fail the stage when the score drops below the threshold you agreed before the change.

Rule: Use Step Functions when the regression run needs retries, a branch for each case, or a human approval before promotion.

Trap: A regression suite that checks only that the API call succeeds misses wrong content. A model can return a valid response that invents a policy detail.

Trap: Built-in datasets test general behavior. Use your own domain cases for a release gate.

Pairs with: CodeBuild with a buildspec file for the test stage, CloudWatch alarms for failed runs, and S3 for stored outputs. **Limits and numbers:** Step Functions concurrency limits are Not in source for this review.

Source: course slide extract, pages 379, 431, and 661. No AWS page was fetched for golden dataset practice itself.

Prompt A/B tests with Prompt Management variants

A Prompt Management prompt can hold several variants to compare.

Each variant has its own name, model or agent resource, template, and inference settings. In the console, compare mode lets you add up to three variants. You can run each variant with the same test variables and read the outputs side by side.

The inference settings include maximum tokens, stop sequences, temperature, and top P. Temperature near zero makes the output less random. The course material also describes top K and top P as two ways to limit the sampling pool.

Rule: Use variants to keep two prompt versions side by side. Then use an evaluation job on both variants with the same dataset to get scores.

Trap: Compare mode is a manual check on a few inputs. It is not a regression gate. The source does not confirm that variants feed a model evaluation job, so treat that link as Not in source.

Trap: A live traffic split tests business outcomes, but it exposes real users to the weaker variant. Use it after an offline gate passes.

Pairs with: AppConfig for changing the active variant without a code change, and CloudWatch Evidently for experiments. **Limits and numbers:** three variants in compare mode, as stated on the console procedure.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-management-create.html and course slide extract, pages 298 and 365

AgentCore Evaluations: online and offline

AgentCore Evaluations scores agent behavior with built-in and custom evaluators.

Built-in evaluators use LLM-as-a-judge scoring on converted traces. The service integrates with the Strands and LangGraph frameworks and with OpenTelemetry and OpenInference instrumentation. Built-in evaluator identifiers have the form Builtin followed by the name, such as Builtin.Helpfulness.

An online evaluation configuration names the evaluators, the data source to watch, and the evaluation parameters. The course material says the sampling setting controls the share of sessions that are scored, and that results can appear in CloudWatch and in AgentCore Observability. The default quota is 1,000 evaluation configurations per Region per account.

Rule: Use online evaluation when the question is about live sessions after release. Use on-demand evaluation for one trace or span. Use batch evaluation for a regression view across many saved sessions.

The course material lists built-in checks for correctness, helpfulness, faithfulness, relevance, coherence, refusal, goal completion, tool choice, parameter checks, and harmful content. It also says evaluation may be limited by filters and the sampling rate you choose.

Trap: Online evaluation needs a deployed agent and live traffic. It cannot approve a candidate before release on its own.

Trap: Guardrails block unsafe output at runtime. They do not score the agent's quality, and they do not replace evaluators.

Pairs with: CloudWatch alarms, AgentCore Observability traces, and the batch and on-demand modes. **Limits and numbers:** the evaluator count per configuration and the sampling rate values are Not in source for this review.

Source: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/evaluations.html and https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/online-evaluations.html and course slide extract, pages 250 and 252

Red teaming

Red teaming sends hostile or edge-case inputs on purpose to find failures.

The course material lists hallucination rate, semantic drift, faithfulness, and compliance as output checks. It also lists Bedrock model evaluation tools, Clarify bias detection, Model Monitor alerts, and A2I as responsible AI tools. Specific AWS red teaming tooling is Not in source for this review.

Rule: Build an adversarial set by category, such as jailbreak, private data request, toxic bait, and off-topic. Score each category separately and report the failure rate per category.

Trap: A red team set made of easy prompts gives a false pass. The set must include the attacks that the team expects.

Trap: A count of blocked requests shows that a control fired. It does not show that the model is safe without the control.

Pairs with: Bedrock Guardrails for runtime controls, judge metrics for scoring, and A2I for human review of borderline cases. **Limits and numbers:** Not in source for red teaming limits.

Source: course slide extract, pages 431 and 433. The definition is a general practice term, not an AWS-sourced fact.

SageMaker Clarify FMEval

Clarify foundation model evaluation runs automatic and human jobs.

Automatic jobs run in SageMaker Studio or with the fmeval library. Human-worker jobs run in Studio. Automatic jobs use one task type each. The four task types are open-ended generation, text summarization, question answering, and classification.

The fmeval library is the path for LLMs outside AWS and for models from other services, according to the AWS page. Studio needs the updated Studio experience. The built-in datasets vary by task, for example TREX and BOLD for generation, Government Report and Gigaword for summarization, and BoolQ, NaturalQuestions, and TriviaQA for question answering.

Clarify also has pre-training bias metrics, such as class imbalance, difference in proportions of labels, and KL or JS divergence. These work on a labeled training dataset, not on generated text.

Rule: Choose Clarify FMEval when you need dimension scores such as factual knowledge, stereotyping, toxicity, or semantic robustness, or when the model runs outside Bedrock.

Trap: Clarify pre-training bias metrics do not score a generated answer. They describe a training dataset.

Trap: A Bedrock-hosted model with a managed console job does not need the fmeval library. Choosing the library adds code work the scenario did not ask for.

Pairs with: Bedrock model evaluation for Bedrock-hosted models, SageMaker Model Monitor for production drift, and S3 for datasets and results. **Limits and numbers:** the per-job prompt count is Not in source for this review.

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-foundation-model-evaluate.html and https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-foundation-model-evaluate-auto.html and https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-foundation-model-evaluate-overview.html and course slide extract, page 328

Human review with A2I

Amazon Augmented AI adds human review to predictions in production.

High-confidence predictions return to the client at once. Low-confidence predictions go to human reviewers. Reviews are consolidated with weighted scores and stored in S3. Reviewers can be your employees, a vendor workforce, or Amazon Mechanical Turk.

Rule: Use A2I when a live output must have a human decision before delivery, based on a confidence or risk rule.

A2I works one item at a time. It is a control in the request path, so it adds latency for the items that go to review. Reviewed items can also be added to a training dataset later.

Trap: A2I is not an offline benchmark. It does not compare two models over a fixed dataset the way a Bedrock human job does.

Trap: Ground Truth labeling creates labels for training data. It is not a live review loop for generated answers.

Pairs with: Bedrock human jobs for offline ratings, Lambda and API Gateway for the request path, and S3 for stored reviews. **Limits and numbers:** the flow definition limits are Not in source for this review.

Source: course slide extract, pages 259, 321, and 594. The AWS A2I product page was not confirmed in this review.

CloudWatch for production quality signals

CloudWatch metrics belong to namespaces and can hold up to 30 dimensions.

You can publish custom metrics, build dashboards, and create alarms. An alarm has three states: OK, INSUFFICIENT_DATA, and ALARM. High-resolution custom metrics can use periods of 10 or 30 seconds, or multiples of 60 seconds. Composite alarms combine several alarms with AND and OR logic.

A metric filter on a log group can turn a log pattern into a metric. A CloudWatch alarm then watches the new metric. The course material lists production signals such as trace data, business results, prompt effect, hallucination trends, token bursts, response drift, invocation logs, and cost anomaly checks.

Rule: Alarm on a score that an evaluator or job already emits as a metric. A raw log line cannot trigger a threshold until a metric filter or a custom metric exists.

Trap: CloudWatch has no built-in fairness metric for generated text, in the source reviewed here. Publish a cohort-level score as your own custom metric.

Trap: Infrastructure metrics such as CPU or memory do not show answer quality. A healthy endpoint can still return wrong answers.

Pairs with: AgentCore online evaluation, Model Monitor for ML drift, and Synthetics canaries for scripted checks. **Limits and numbers:** the default

AgentCore metric namespace is stated in the AgentCore guidance but was not confirmed on the fetched page, so treat it as Not in source.

Source: course slide extract, pages 109, 110, 440, 442, 444, and 446

Troubleshooting by symptom

Start from the symptom the user reports. Then pick the first check. Each symptom has a first check that is cheap and a deeper check that costs more. Do the cheap check first.

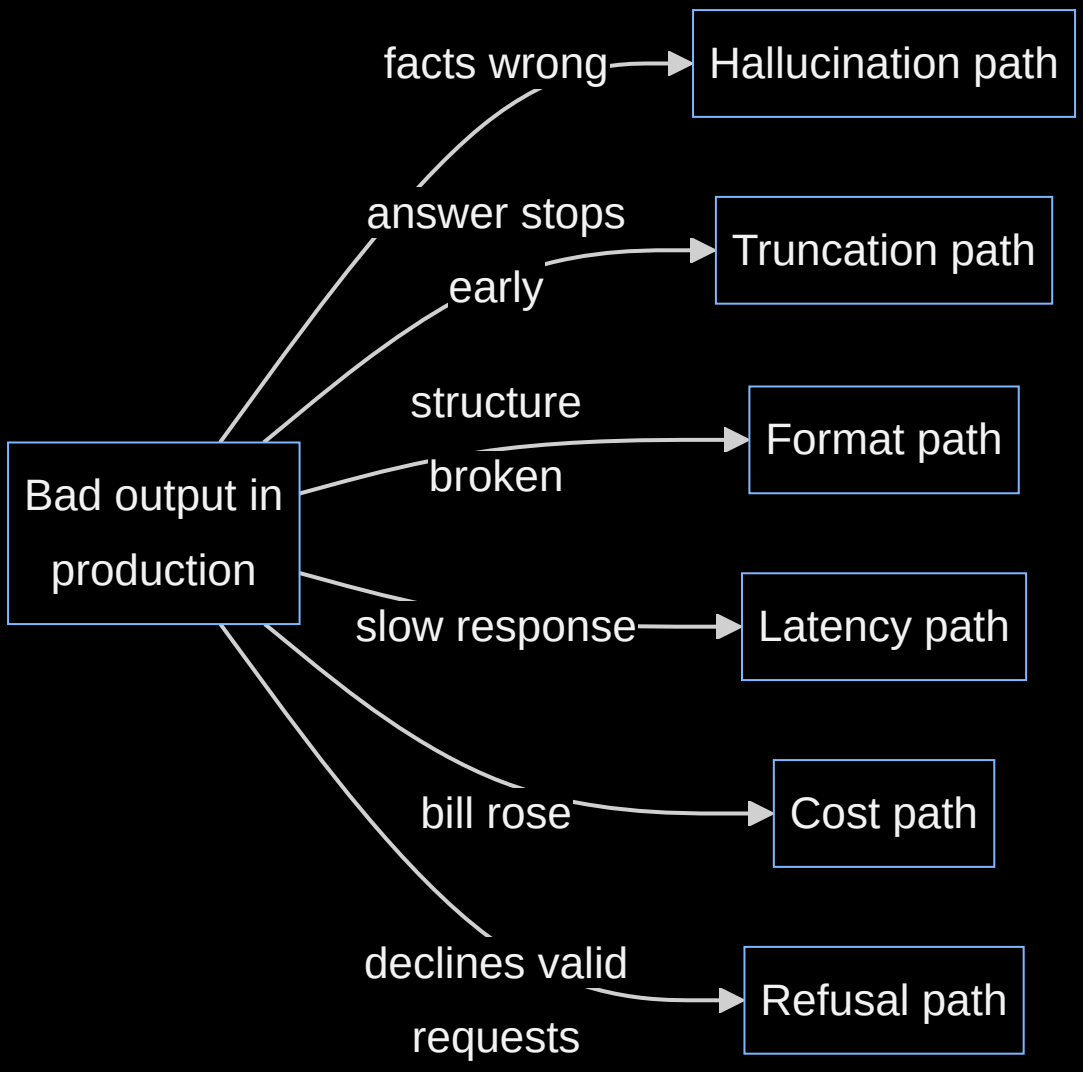


Figure: Match the symptom to one path, then run its first check.

Hallucination

Faithfulness checks whether an answer stays inside the retrieved context.

The course material lists hallucination rate and faithfulness as output checks. Temperature changes randomness, but it does not add facts that are missing from the context.

Rule: For a RAG app, run a retrieve-only job first. If the passages are wrong, fix retrieval before you change the generator.

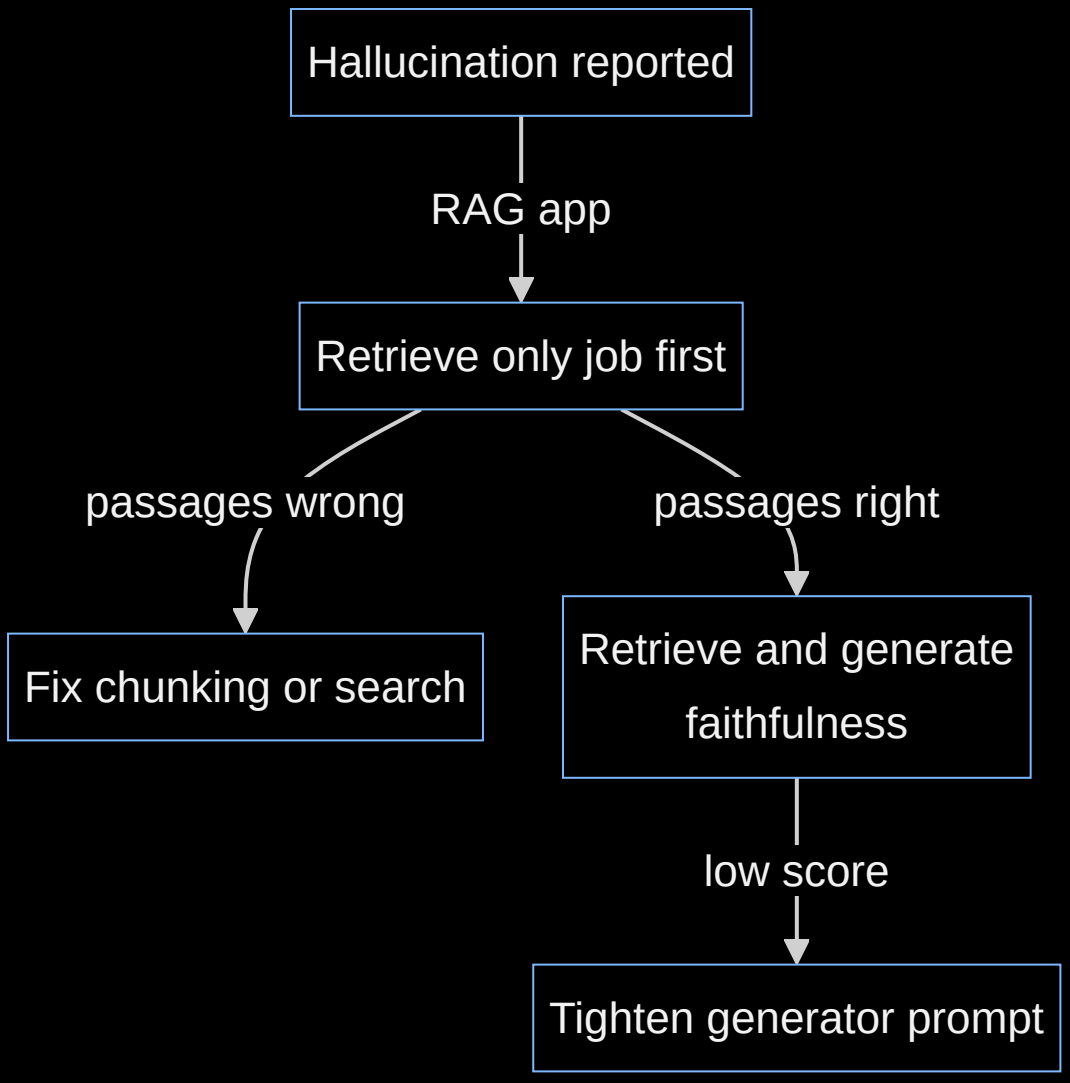


Figure: Hallucination work starts with retrieval, then moves to the generator.

Trap: Lowering temperature to zero does not remove a wrong fact that the context also contains. It only reduces variation between runs.

Truncation

In Prompt Management, the maximum tokens setting limits the length of the generated response. Stop sequences end generation when a listed text appears. Both settings are on the prompt or the variant.

Rule: For a cut-off answer, check the maximum tokens value and the stop sequences before you change the prompt wording.

Trap: A higher temperature does not stop an early end. It changes word choice, not the length limit.

Wrong format

A judge metric for following instructions scores whether the response respects the directions in the prompt. A structure check is a separate test. It parses the output, for example as JSON, and checks the fields.

Rule: Add a parse and schema check in code after generation, and run it in the golden dataset on every change.

Trap: A judge can say the format is acceptable when the output has a small syntax error. Use a parser for structure, and use the judge for intent.

Latency

The course material lists foundation model interaction tracing and invocation logs as production signals. X-Ray and trace data show where time goes across the components. The specific latency metric names for Bedrock invocations are Not in source for this review.

Rule: Measure latency per step with traces before you change the model. A slow retrieval step needs a different fix than a slow generation step.

Trap: A quality evaluation does not measure response time. A faster model can still fail the quality gate.

Cost spike

The course material links cost to token counts. It suggests token counting with the price performance ratio, and it lists token burst patterns and cost anomaly detection as signals to watch.

Rule: For a cost spike, compare input and output token counts per request type before you change the model.

Trap: Moving to a larger model rarely lowers cost. It usually raises the price per token, so check the token growth first.

Refusals

The built-in judge metric for refusal determines whether the response declines the prompt or rejects the request with reasons. Guardrails can also block an

answer. The two signals are different, so read both.

Rule: Decide the expected behavior for each test case first. A refusal is correct when the golden case expects a refusal, and wrong when the case expects an answer.

Trap: A high refusal rate is not always a fault. A correct policy refusal scores the same as an over-cautious one unless the expected label is in the dataset.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/model-evaluation-metrics.html and https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-management-create.html and course slide extract, pages 289, 298, 431, and 446

X versus Y pairs

Each pair below has one decision tree, a three-row comparison, one question to ask yourself, and the trap. Read the question first, then use the tree to answer it. The three rows are purpose, input, and output or effect.

Bedrock model evaluation versus SageMaker Clarify FMEval

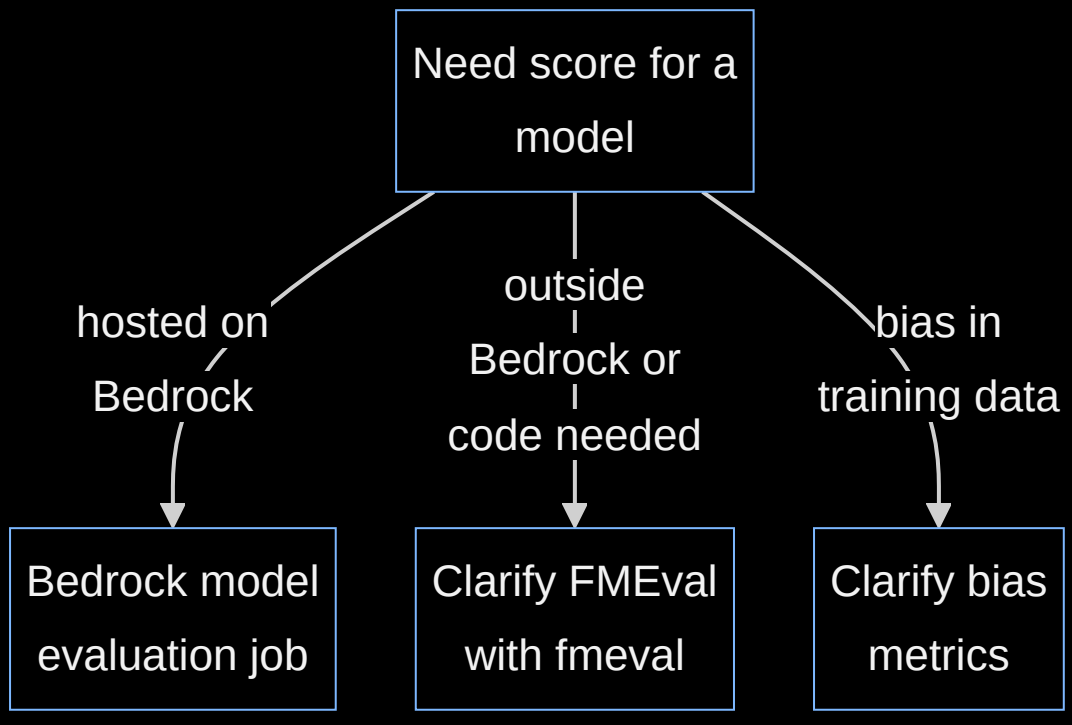


Figure: Hosting location and code need decide the first split.

- **Purpose:** Bedrock gives a managed job for Bedrock models. Clarify gives dimension scores and code control.
- **Input:** Bedrock takes a prompt dataset in S3. Clarify takes a built-in dataset or your own prompts, through Studio or fmeval.
- **Output:** Bedrock writes a report with metric scores and judge explanations. Clarify writes results per dimension to S3.

Exam tip: Ask yourself, is the model hosted on Bedrock, and does the team want a managed console job without scoring code.

Trap: Choosing Clarify because it has the word evaluation in the name. A Bedrock-hosted model with a managed need points to the Bedrock job.

Built-in metrics versus LLM as a judge

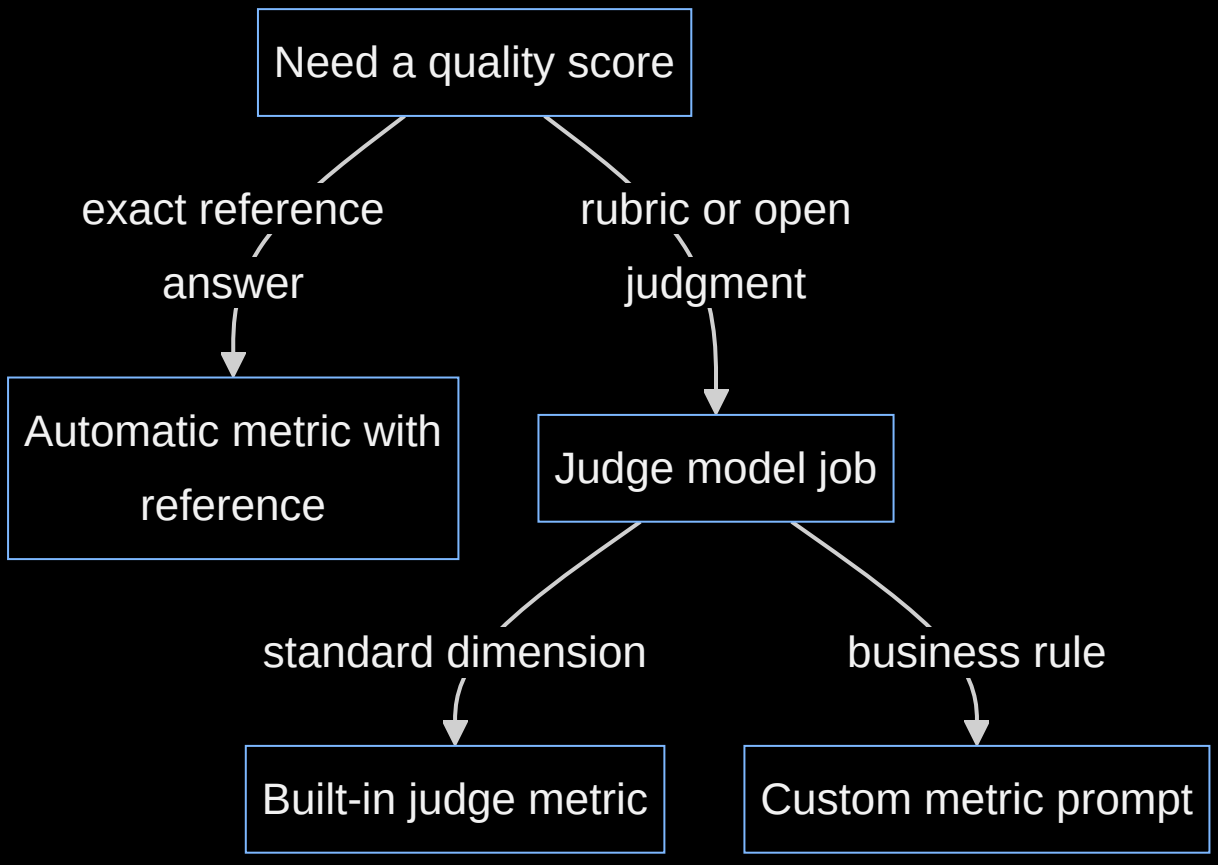


Figure: A reference string favors metrics, and a rubric favors a judge.

- **Purpose:** Automatic metrics measure overlap or accuracy against a reference. A judge scores quality against a rubric.
- **Input:** Automatic metrics need references for many tasks. A judge needs a prompt and a response, and optionally a reference.

- **Output:** Automatic metrics give a number per prompt. A judge gives a score and an explanation.

Exam tip: Ask yourself, is there one correct string, or is quality a matter of judgment.

Trap: A judge is not free of cost or bias. The source notes the judge can share the generator's blind spots.

Bedrock human job versus A2I review

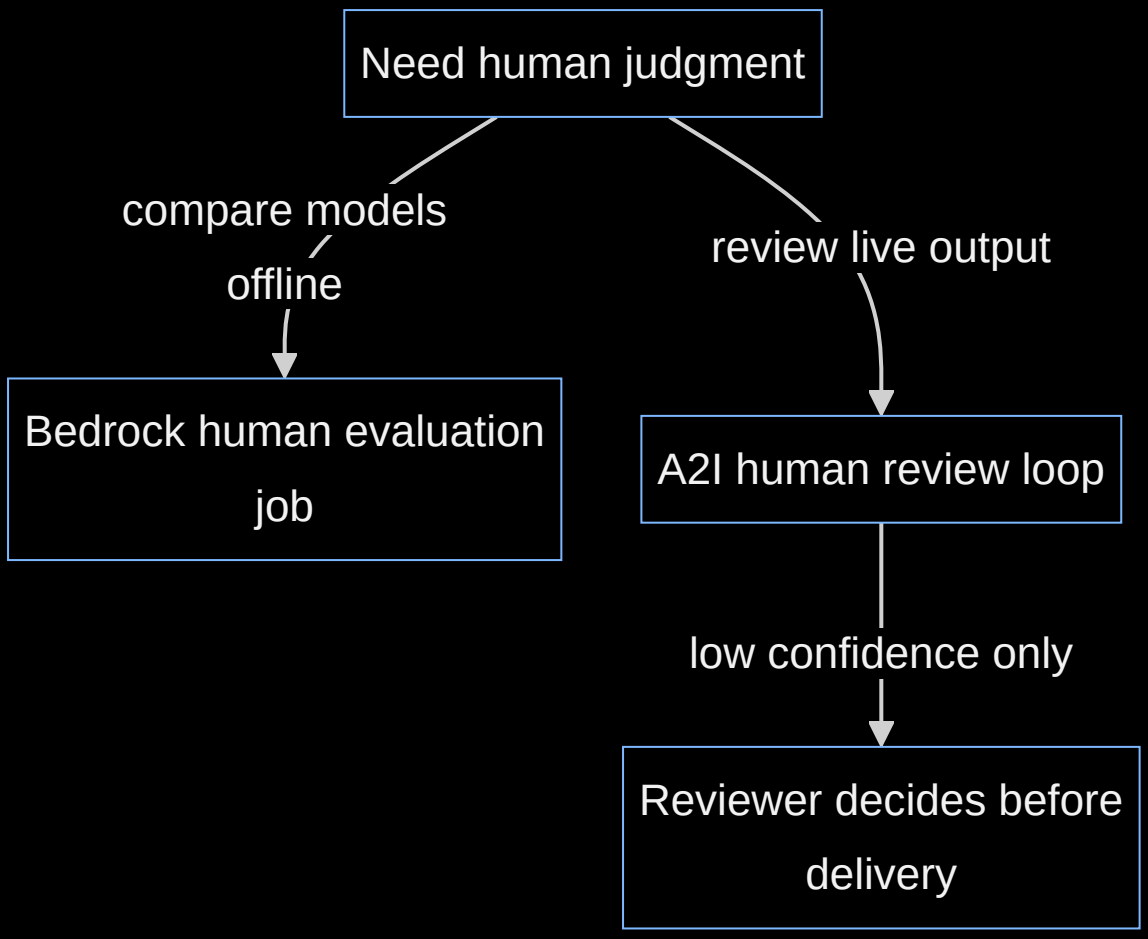


Figure: Offline comparison and live review are different workflows.

- **Purpose:** A Bedrock human job rates models on a dataset. A2I routes live items to people when the confidence is low.
- **Input:** The Bedrock job takes a prompt dataset and a work team. A2I takes a prediction and a confidence rule.
- **Output:** The Bedrock job gives ratings per model. A2I gives a reviewed result and stores the review in S3.

Exam tip: Ask yourself, does the human see the output before a user does, or only during a release check.

Trap: Using A2I for a release comparison of two models. It handles items as they arrive, so it is not a fixed benchmark.

Retrieve only versus retrieve and generate

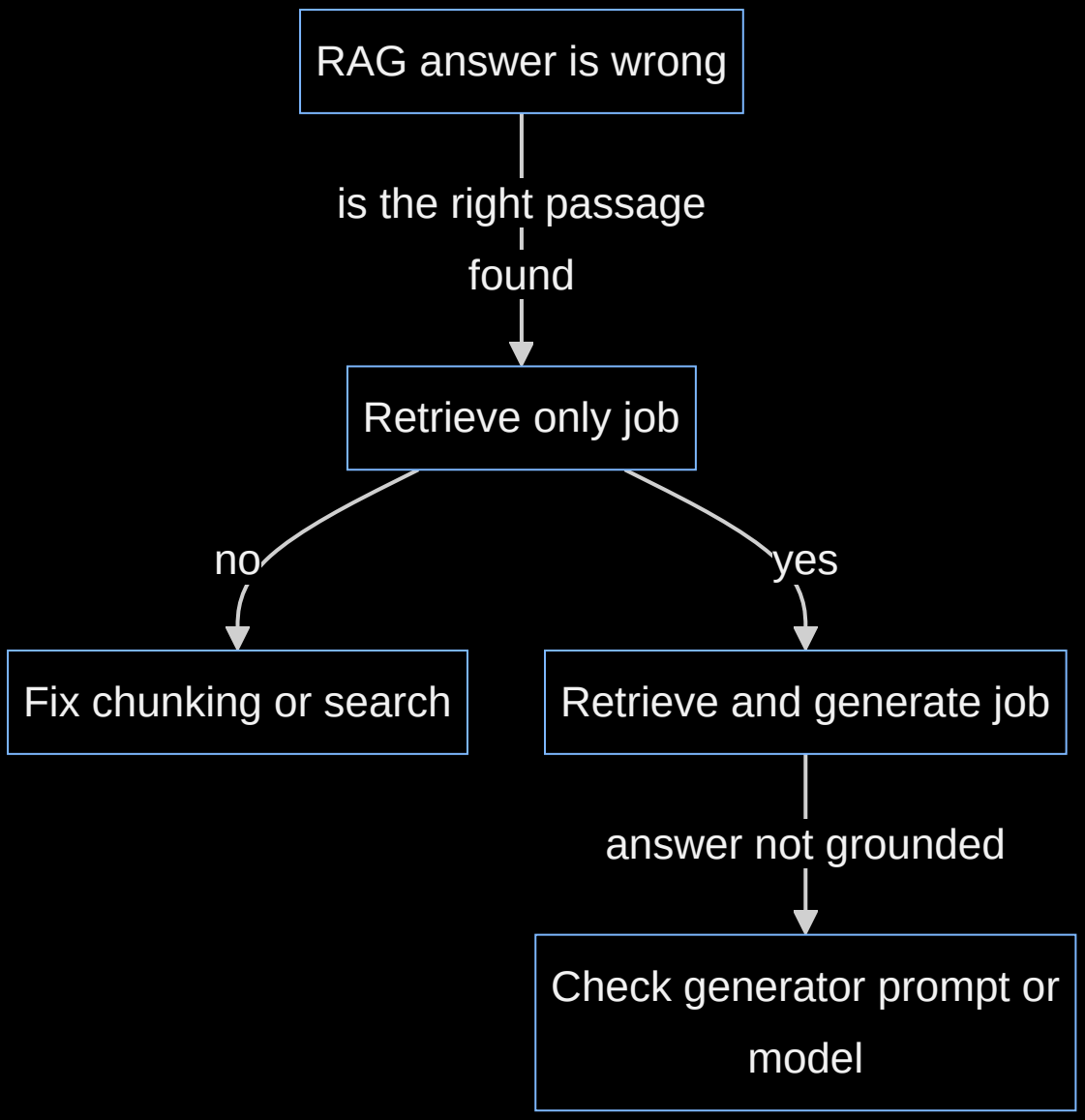


Figure: Retrieval is tested before generation.

- **Purpose:** Retrieve only checks the passages. Retrieve and generate checks the passages and the final answer.
- **Input:** Retrieve only needs queries and passage references. Retrieve and generate needs queries, references, and a generator model.
- **Output:** Retrieve only reports retrieval metrics. Retrieve and generate reports answer metrics as well.

Exam tip: Ask yourself, does the question change the search step or the answer step.

Trap: Running only a retrieve and generate job when the team changed the chunk size. The answer score mixes two causes and hides the search effect.

Online versus offline agent evaluation

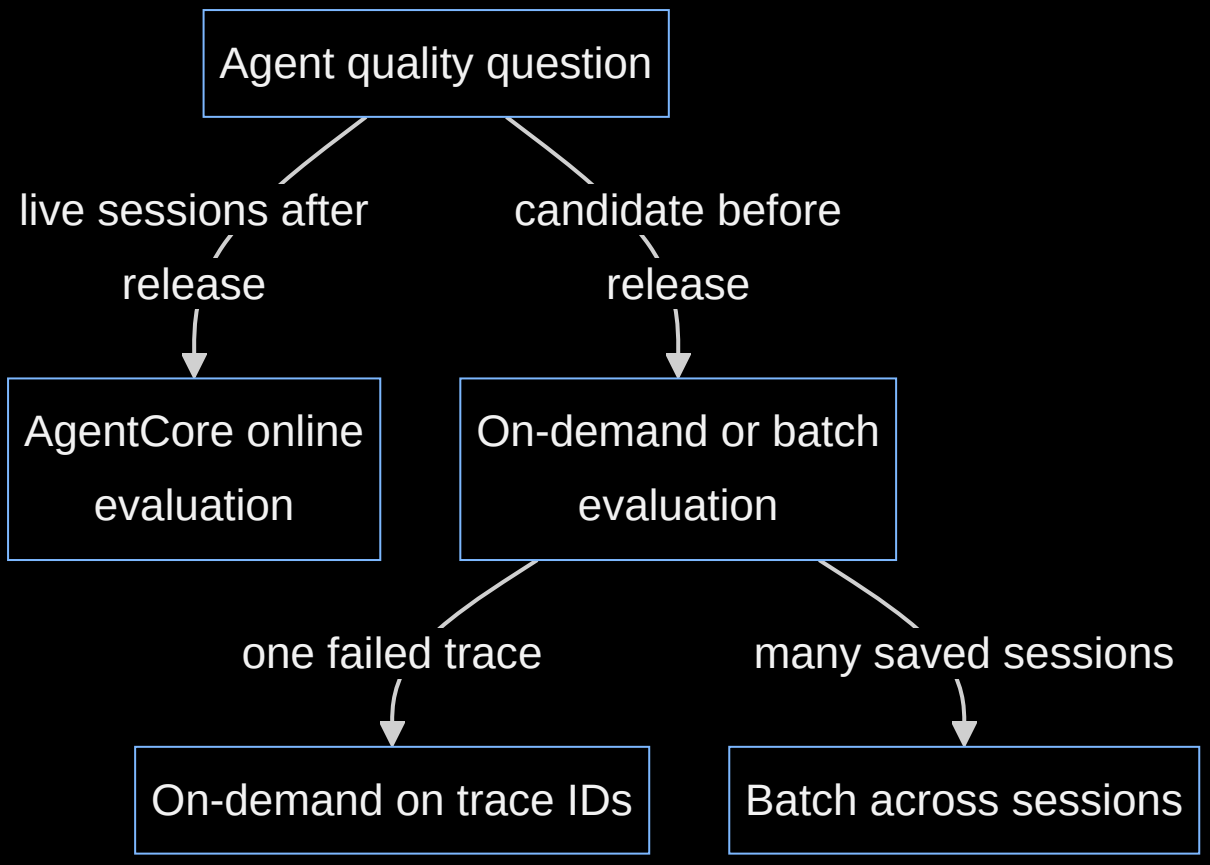


Figure: Timing and scope decide online or offline.

- **Purpose:** Online watches live traffic with sampling. Offline checks a candidate or investigates saved traces.
- **Input:** Online reads the live data source. Offline reads saved sessions, traces, or spans.
- **Output:** Online gives continuous scores for alarms. Offline gives a point-in-time result for a release decision.

Exam tip: Ask yourself, is the question about a release gate or about what users see now.

Trap: Using online evaluation as the only pre-release test. It needs the deployed agent and live traffic.

ROUGE versus BLEU

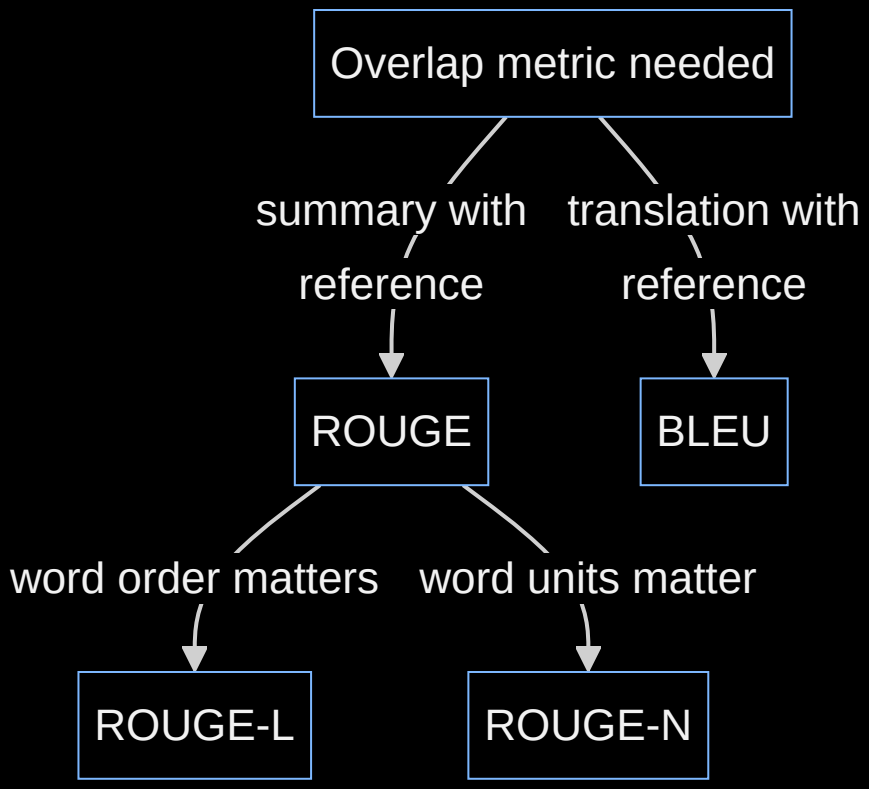


Figure: The task decides the metric family.

- **Purpose:** ROUGE measures recall of reference units in summaries. BLEU measures precision of n-grams in translations.
- **Input:** Both need a generated text and at least one reference text.
- **Output:** ROUGE gives overlap scores per unit type. BLEU gives a precision score with a brevity penalty.

Exam tip: Ask yourself, is the task summary or translation, and is the reference recall or precision.

Trap: BLEU for a summary task. The brevity penalty and precision focus do not match a summary goal.

BERTScore versus n-gram metrics

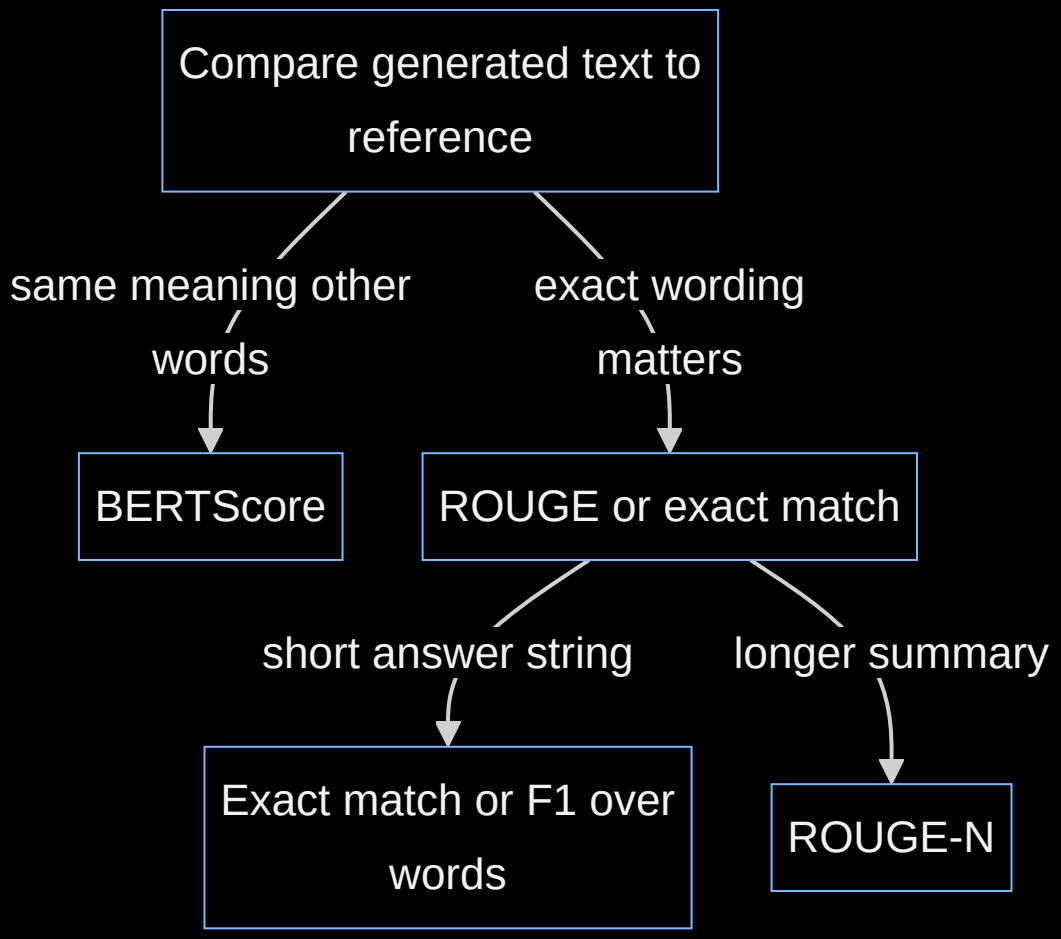


Figure: Meaning favors embeddings, and wording favors overlap.

- **Purpose:** BERTScore measures semantic similarity with embeddings. N-gram metrics measure shared words or sequences.
- **Input:** Both need a generated text and a reference.
- **Output:** BERTScore gives a similarity score. N-gram metrics give overlap counts or precision and recall.

Exam tip: Ask yourself, must the wording match, or only the meaning.

Trap: Calling BERTScore a fact check. A similar meaning score can miss a changed number.

Model Monitor versus evaluation jobs

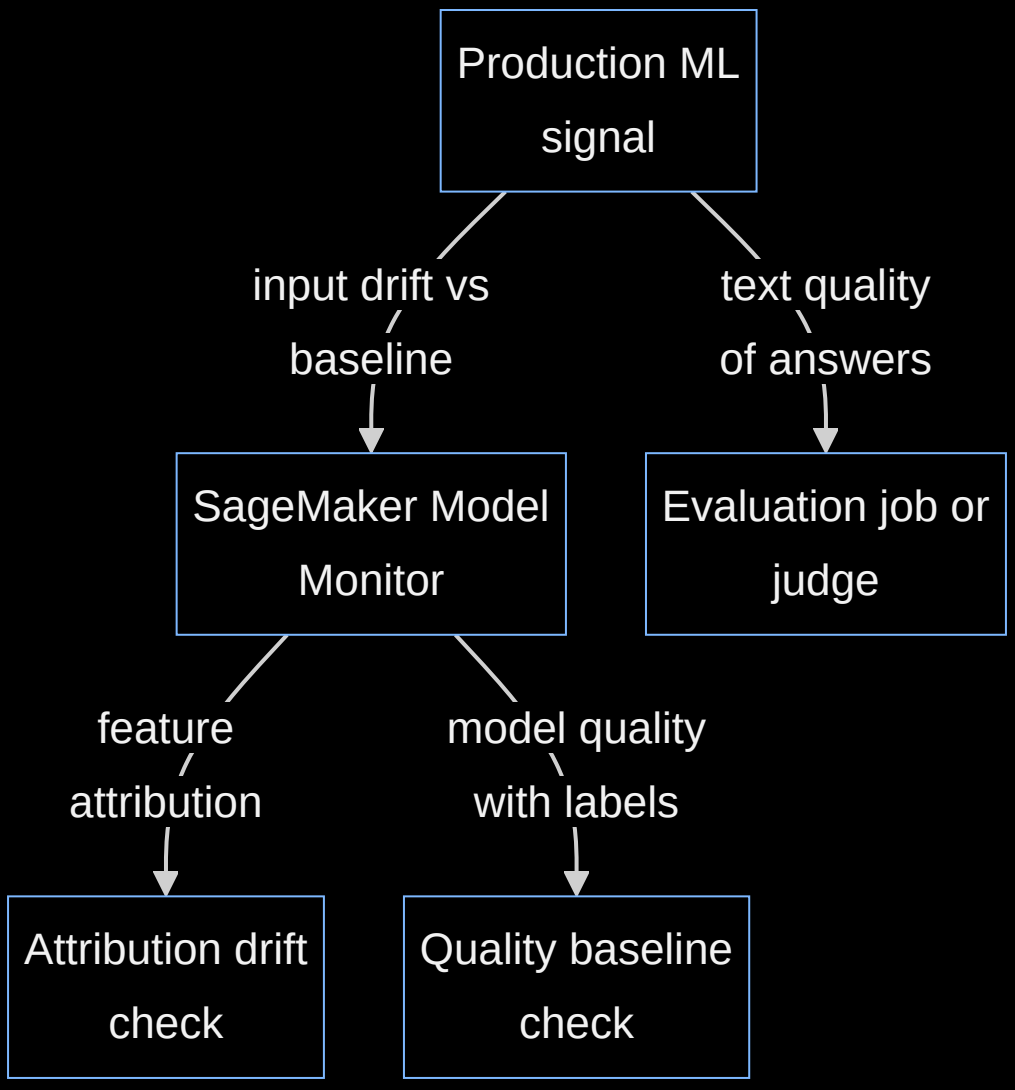


Figure: Monitoring tracks statistics, and evaluation scores answers.

- **Purpose:** Model Monitor watches data and model statistics against a baseline. Evaluation scores the content of generated answers.
- **Input:** Model Monitor reads captured data in S3 on a schedule. Evaluation reads prompts, responses, and references.
- **Output:** Model Monitor emits metrics to CloudWatch for alarms. Evaluation gives scores and explanations per prompt.

Exam tip: Ask yourself, does the question ask about the statistics of inputs, or about whether the text is right.

Trap: Model Monitor as the answer for free-form text quality. The course material says its quality checks use statistical properties of features and labels.

Golden dataset regression versus canary or shadow test

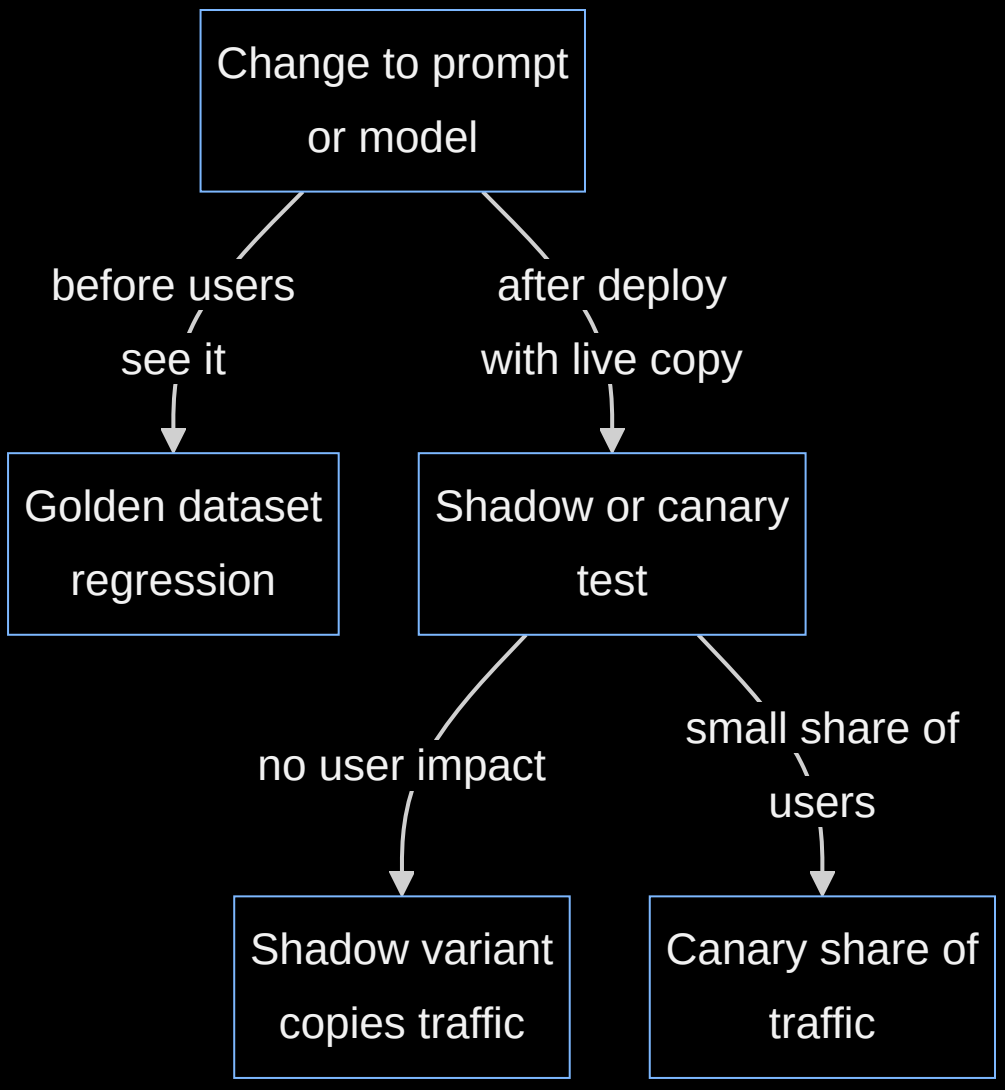


Figure: Pre-release gates and live tests answer different questions.

- **Purpose:** A golden run gates a release on fixed cases. A shadow or canary compares the candidate to the live model on real traffic.
- **Input:** A golden run needs a fixed dataset. A shadow or canary needs live requests and a routing setting.
- **Output:** A golden run gives pass or fail per threshold. A shadow or canary gives comparison data on real requests.

Exam tip: Ask yourself, is the test before any user sees the change, or with real users at a small share.

Trap: A canary as the only regression gate. Users see the candidate's errors before the team knows the score.

Guardrails versus evaluation

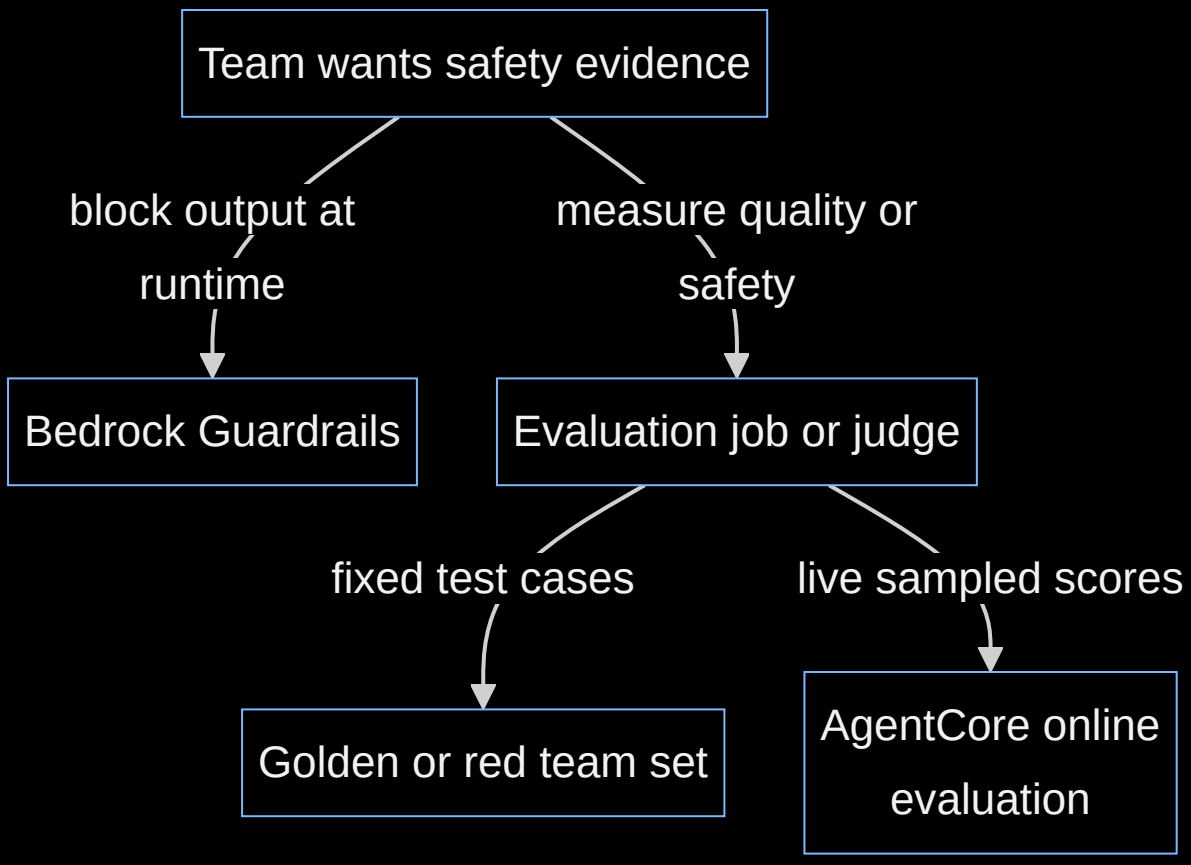


Figure: Guardrails act on traffic, and evaluation produces scores.

- **Purpose:** Guardrails enforce rules at runtime. Evaluation measures how well the system behaves.
- **Input:** Guardrails read each request and response. Evaluation reads a dataset or sampled sessions.
- **Output:** Guardrails block or allow. Evaluation gives scores over time.

Exam tip: Ask yourself, does the team need a control on each request, or a score for the system.

Trap: Guardrails as proof of quality. A block count shows the control fired, not that the answers were correct.

Prompt variants versus traffic split

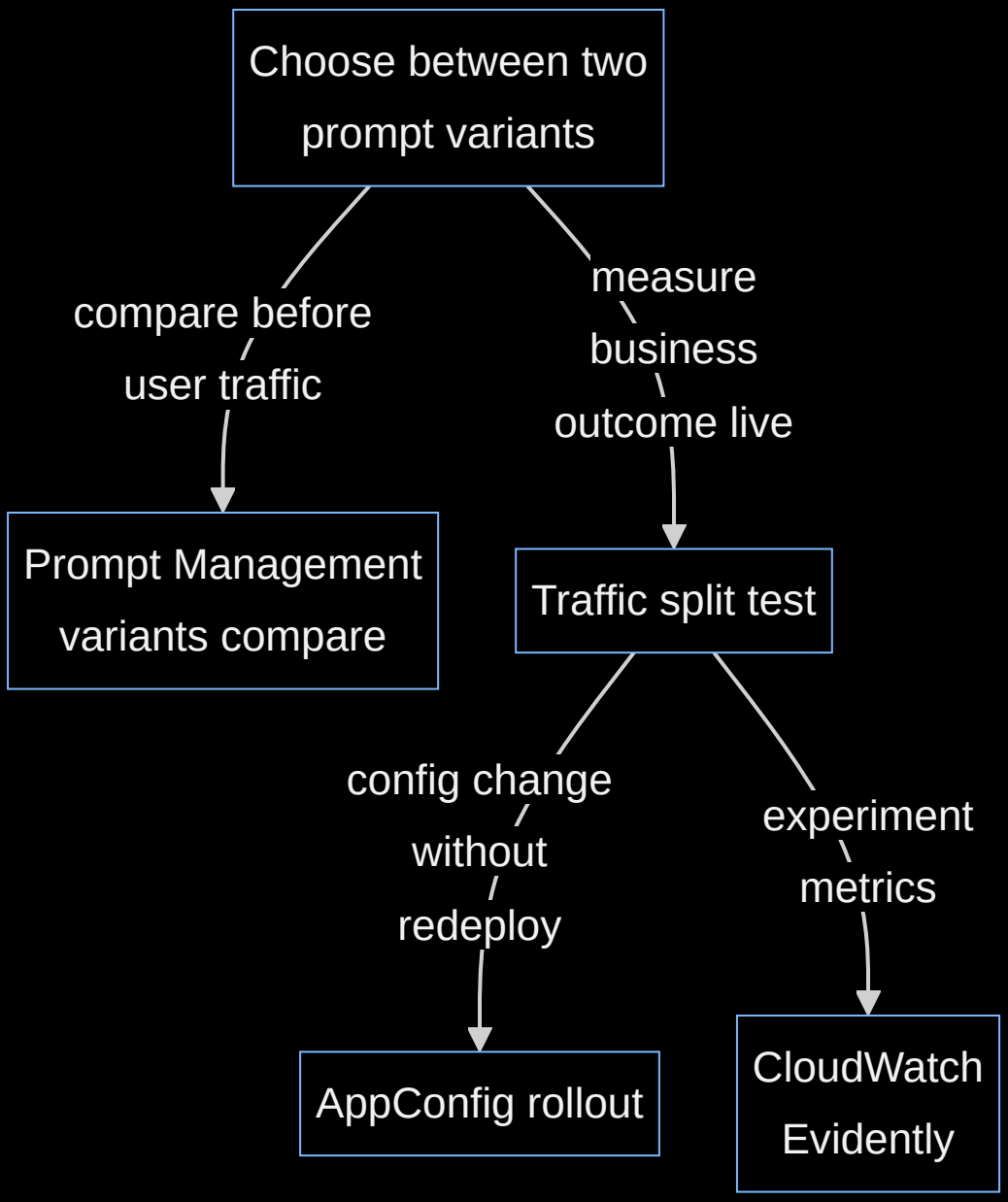


Figure: Compare offline first, then split live traffic.

- **Purpose:** Variants compare outputs side by side. A traffic split changes what real users receive.
- **Input:** Variants need test inputs. A traffic split needs a routing rule and metrics.
- **Output:** Variants show outputs for review. A traffic split shows business and quality metrics per arm.

Exam tip: Ask yourself, is the decision made before any user sees the variant, or from live results.

Trap: Using a live split to pick a prompt when the team has no offline test. The weaker variant reaches real users first.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-management-create.html and course slide extract, pages 298, 313, 330, 365, 430, and 440. Pair content is drawn from the AWS and course material above.

Read a long question

Long questions hide the deciding constraint in one clause. Read the last sentence first, to learn the ask. Then read the middle for the constraint. Bold words in the scenarios below mark the constraint. Each scenario is written for this module and is not taken from any test.

Scenario one: a weekly model swap with no reviewers

A claims team summarizes claim files with a Bedrock model. Each week, the team wants to test a different model before a release. They have 400 files, each with an approved summary, and no reviewer hours this month.

Constraints: **no reviewer hours**, **approved summary**, and **each week**. The first two point to a reference-based automatic job. The third points to a repeatable run that a pipeline can start.

Eliminate the human workforce job, because it needs reviewer hours. Eliminate A2I, because it reviews live items and does not compare models. Eliminate the fmeval library, because the models are Bedrock-hosted and the team wants no scoring code. Eliminate the traffic split, because it tests live users and skips the offline gate.

Rule: Answer with a Bedrock automatic evaluation job that uses the approved summaries as references, run from the weekly pipeline.

Scenario two: fluent answers that cite wrong steps

A support assistant answers questions from a set of product manuals through a knowledge base. Users say the answers read well but cite the wrong steps. The

team has a fixed list of 200 questions with known correct manual sections, and it wants to know whether the search or the answer step fails.

Constraints: **wrong steps**, **known correct sections**, and **search or answer**. The known sections make retrieval testable. The phrase search or answer asks you to split the stages.

Eliminate the CloudWatch dashboard, because it shows trends and not the cause. Eliminate a model swap, because it changes the generator without isolating the search step. Eliminate human review of live answers, because it is slow and does not split the stages. Eliminate a retrieve and generate job alone, because the score mixes search and answer causes.

Rule: Run a retrieve-only job on the fixed list first. Then run a retrieve and generate job on the same list if the passages are right.

Scenario three: a wrong tool call in live bookings

An agent books trips and calls tools to change reservations. After release, the team sees a few bookings with the wrong tool, and it wants a daily sampled score with alarms. The team cannot change the agent code this sprint.

Constraints: **after release**, **daily sampled score**, **alarms**, and **cannot change agent code**. Sampling and alarms point to an online evaluation with a metric in CloudWatch.

Eliminate the offline judge job on exported transcripts, because it is not live and needs a daily export step. Eliminate Guardrails, because they block output and do not score tool choice. Eliminate a Synthetics canary with a scripted check, because a script cannot judge the tool choice in a free-form session. Eliminate Model Monitor, because the question is about agent behavior, not feature statistics.

Rule: Configure an AgentCore online evaluation with the tool selection evaluator, sample the sessions, and alarm on the published score.

Source: course slide extract, pages 250 and 440, and https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/online-evaluations.html

Distractor patterns seen in the tests

Wrong options in this area follow a few patterns. Learn the pattern, and you can reject a wrong option even when you do not know the exact service. Each pattern below has a reason it is wrong.

- Trap: Right service, wrong stage. A pre-release tool is offered for live monitoring, or a live tool is offered for a release gate. It is wrong because the timing does not match the question.
- Trap: Metric from another task. A translation metric is used for summaries, or a classification metric is used for free-form answers. It is wrong because the metric does not measure the task output.
- Trap: Safety control offered as evaluation. A content filter or guardrail is offered as the way to measure quality. It is wrong because it blocks at runtime and does not score.
- Trap: Infrastructure signal offered as quality. CPU, memory, or uptime is offered to show answer correctness. It is wrong because it measures the server, not the text.
- Trap: Manual effort that does not scale. A team of reviewers or developers writing tests by hand is offered as the fix. It is wrong because the scenario asks for a repeatable, automated check.
- Trap: Labeling tool for generated output. A labeling service is offered to score answers. It is wrong because labeling creates training data and does not run a review loop for generated text.
- Trap: Custom build when a managed feature fits. A custom judge pipeline with new endpoints is offered when a managed job would do. It is wrong unless the constraint says the managed job cannot run the model.
- Trap: Cost tool offered as quality. A pricing or rightsizing tool is offered to pick the better model. It is wrong because it measures spend, not output quality.
- Trap: Fixing the wrong stage. The generator is changed when the search returned the wrong passages. It is wrong because the cause sits upstream.

- Trap: Limit that breaks the count. A human job or a dataset is sized beyond a documented limit. It is wrong because the design cannot run as stated. Check the count against the limit in the source.
- Trap: Average hides a group. One overall score is offered as proof of fairness. It is wrong because a strong average can hide a weak group. Report scores per cohort.
- Trap: Judge checked by itself. A judge model from the same family as the generator is offered as neutral. It is wrong because shared blind spots can pass a bad answer.

Drill

Each prompt is one line. Cover the answer line, answer in your head, then check. Each answer starts with Rule or Trap.

1. Two models summarize files that have approved summaries, and no reviewer is available. Which job fits?

Rule: A Bedrock automatic evaluation job with the approved summaries as references.

2. The team needs a metric that flags answers with claims absent from the supplied context.

Rule: Faithfulness, which is Builtin.Faithfulness on the judge list.

3. A team uses a guardrail denied topic list as its hallucination score.

Trap: A guardrail blocks content at runtime. It does not score answer grounding.

4. A new prompt drops the structured output format on some inputs. Where is it caught before release?

Rule: A golden dataset run in the pipeline, with a parse and schema check on each output.

5. Answers from a knowledge base are wrong, and the team does not know if search or generation fails.

Rule: Run a retrieve-only job first to test the passages.

6. A deployed agent needs a daily sampled score for correctness and tool use, with alarms.

Rule: An AgentCore online evaluation with alarms on the published scores.

7. A low-confidence model result must reach a person before a customer sees it.

Rule: Amazon Augmented AI human review on the low-confidence items.

8. Thirty staff must rate two candidate models on a fixed prompt set.

Rule: A Bedrock human evaluation job, with the work team under the stated worker limit.

9. A translation team wants a precision-oriented score against human translations.

Rule: BLEU, because it measures n-gram precision against references.

10. Summaries use different words from the reference but keep the same meaning, and ROUGE scores are low.

Rule: BERTScore, because it compares embeddings for meaning.

11. Two prompt versions must be compared on a few inputs before any test run.

Rule: Prompt Management variants in compare mode, up to three variants.

12. Responses end in the middle of a sentence after a prompt change.

Rule: Check the maximum tokens value and the stop sequences first.

13. A model declines valid policy questions, and the team wants to measure this.

Rule: Use the refusal judge metric, and compare each case with the expected answer or refusal in the golden set.

14. A monthly bill jumps after a prompt change, with no change in traffic.

Rule: Compare input and output token counts per request type before you change the model.

Sources

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation-judge.html

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/model-evaluation-metrics.html

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation-kb.html

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/model-evaluation-type-automatic.html

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/human-worker-evaluations.html

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-management-create.html

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-foundation-model-evaluate.html

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-foundation-model-evaluate-auto.html

Source: https://docs.aws.amazon.com/sagemaker/latest/dg/clarify-foundation-model-evaluate-overview.html

Source: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/evaluations.html

Source: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/online-evaluations.html

Source: course slide extract, module eight pages as cited in each section above.